Algoritmos y Estructuras de Datos - DC - UBA

Segundo cuatrimestre de 2026

TP2: Implementación de TAD: Sistema de Fidelización AirMiles

Normativa

Revisión de mitad de TP: viernes 30 de Octubre, en el horario de la clase.
Límite de entrega: domingo 8 de Noviembre a las 23:59.
Defensa del TP: viernes 20 de Noviembre, en el horario de la clase.
Límite de re-entrega: domingo 22 de Noviembre a las 23:59.
Forma de entrega: Subir a la tarea del campus un único archivo comprimido (.zip) que contenga:

- El código fuente en Java.
- Un archivo PDF (`justificacion.pdf`) fundamentando detalladamente por qué su diseño y combinación de estructuras de datos cumplen con los órdenes de complejidad temporal exigidos para cada operación.

Enunciado

En el Trabajo Práctico 1 especificaron formalmente el funcionamiento del programa de millas *AirMiles*. En este Trabajo Práctico deberán implementar dicha especificación en el lenguaje Java, garantizando una alta eficiencia temporal en las operaciones.

Para lograr los órdenes de complejidad requeridos, deberán diseñar e implementar una combinación de estructuras de datos. Pueden utilizar libremente las estructuras de datos desarrolladas durante los Talleres (ABB, Trie, MaxHeap, etc.).

Importante: No está permitido utilizar implementaciones de la biblioteca estándar de Java para estas estructuras de datos complejas (por ejemplo, `java.util.HashMap`, `java.util.TreeMap`, `java.util.PriorityQueue`, `java.util.LinkedList`). Al implementarlas, estas estructuras deben ser genéricas.

Deben utilizar el archivo comprimido `plantilla_alumnos.zip` como proyecto de base para su implementación, el cual ya contiene la interfaz del sistema y los tests iniciales.

Complejidades Requeridas

A continuación se listan las operaciones que deben implementar de la interfaz `AirMiles`, indicando la complejidad temporal esperada. Salvo que se indique lo contrario las complejidades se refieren al **peor caso**.

Sea U la cantidad de usuarios registrados en el sistema y $|L|$ la longitud del nombre de una promoción.

- **crearSistema**: Crea una instancia vacía del sistema. **Complejidad**: $\mathcal{O}(1)$
- **registrarUsuario / registrarUsuarioConReferidor**: Agrega un nuevo usuario (con o sin referidor). **Complejidad**: $\mathcal{O}(\log U)$ en el caso promedio y $\mathcal{O}(U)$ en el peor caso
- **acumularMillas**: Un usuario realiza un vuelo con cierta distancia bajo el nombre de una promoción. **Complejidad**: $\mathcal{O}(\log U + |L|)$ en el caso promedio y $\mathcal{O}(U + |L|)$ en el peor caso
- **canjearMillas**: Un usuario canjea una cantidad de millas por un premio. **Complejidad**: $\mathcal{O}(\log U)$ en el caso promedio y $\mathcal{O}(U)$ en el peor caso
- **transferirMillas**: Transfiere millas desde el saldo disponible de un usuario origen a un usuario destino. **Complejidad**: $\mathcal{O}(\log U)$ en el caso promedio y $\mathcal{O}(U)$ en el peor caso
- **consultarSaldo / consultarCategoria**: Devuelve las millas disponibles o la categoría del usuario. **Complejidad**: $\mathcal{O}(\log U)$ en el caso promedio y $\mathcal{O}(U)$ en el peor caso
- **iniciarPromocion / iniciarPromocionConTope**: Inicia una nueva promoción. **Complejidad**: $\mathcal{O}(|L|)$
- **finalizarPromocion**: Elimina una promoción activa por nombre. **Complejidad**: $\mathcal{O}(|L|)$
- **elMasAcumulador / elMasCanjeador**: Devuelve el usuario con más millas acumuladas o gastadas en la historia. **Complejidad**: $\mathcal{O}(1)$

- **rankingMillas**: Devuelve una lista con los IDs de los N usuarios con más millas disponibles. **Complejidad**: $\mathcal{O}(N \log U)$
- **paresTransferidoresExclusivos**: Devuelve una lista con todos los pares de usuarios que han transferido millas exclusivamente entre sí. **Complejidad**: $\mathcal{O}(U)$